

Smart Travel Agent

Full Framework Documentation

MERN-style · React + Express + Firestore + Gemini

Project: Smart Travel & Route Optimization Agent
Stack: MERN-style (React + Express + Node.js + Firestore)
Date: July 2026

1. High-Level Architecture

This is a MERN-style agentic application where MongoDB is replaced with Cloud Firestore.

Architecture Layers

Layer	Technology	Role
Frontend	React + Vite	User interface, forms, option cards
Backend	Node.js + Express	Agent logic, API routes, orchestration
Database	Firebase Firestore	Persistent agent session state
LLM	Google Gemini (@google/genai)	Optional final-plan enrichment
Auth/Keys	Service account JSON + .env	Firebase + Gemini credentials

Data Flow

User (Browser)	
React UI (port 5173)	
Express API (port 3001)	
Agent Loop	! Destination Parser ! Plan Builder
Firestore	Gemini API (optional)

2. Project Structure

agentic ai/	
server.js	# Express backend + agent brain
package.json	# Backend dependencies
.env	# Secrets (GEMINI_API_KEY, Firebase path)
serviceAccount.json	# Firebase Admin credentials
client/	# React frontend
src/	
App.jsx	# Main UI component
api.js	# HTTP calls to backend
App.css	# Component styles
main.jsx	# React entry point
vite.config.js	# Dev server + API proxy to :3001

3. End-to-End User Request Flow

Step-by-Step

- User enters goal: "Plan a trip to Udaipur under ₹15,000"
- React calls POST /api/agent/run
- Express creates/loads Firestore session
- Agent runs ReAct steps (Thought ! Action ! Observation)
- Destination parser identifies Udaipur
- Conflict detector finds budget "₹15,000"

7. Agent pauses with status: AWAITING_USER_INPUT
8. React shows Option A and Option B cards
9. User clicks Option A
10. React calls POST /api/agent/respond
11. Backend builds Udaipur itinerary
12. Optional single Gemini call enriches the plan
13. Firestore updated to COMPLETED
14. Final itinerary returned to UI

Sequence (Conflict Path)

```
User !' React !' POST /run !' Firestore (create session)
      !' ReAct loop
      !' Conflict detected
      !' Firestore (AWAITING_USER_INPUT)
      !' Return options

User picks option !' React !' POST /respond !' Firestore (log choice)
      !' Build plan
      !' Gemini (optional)
      !' Firestore (COMPLETED)
      !' Return finalPlan
```

4. Backend Deep Dive (`server.js`)

4.1 Startup & Middleware

On boot, the server:

1. Loads .env via dotenv
2. Initializes Firebase Admin !' Firestore connection
3. Initializes Google GenAI (if GEMINI_ENABLED=true and key exists)
4. Sets up Express with cors() + express.json()

4.2 Firestore Session State

Every agent run is stored at sessions/{sessionId}:

```
{
  "sessionId": "session-123",
  "userId": "demo-user",
  "userGoal": "Plan a trip to Udaipur under 15000",
  "status": "AWAITING_USER_INPUT",
  "logs": [],
  "history": [],
  "pendingOptions": [],
  "conflictReason": "...",
  "finalPlan": null,
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

4.3 Status Lifecycle

```
[new] !' RUNNING
RUNNING !' AWAITING_USER_INPUT (budget conflict)
AWAITING_USER_INPUT !' RUNNING (user responds)
RUNNING !' COMPLETED (plan finalized)
COMPLETED !' COMPLETED (idempotent re-fetch)
```

4.4 Route 1: POST /api/agent/run

Input: { userId, sessionId, userGoal }

Behavior:

1. Load or create Firestore session
2. Run ReAct mock loop
3. Parse destination from goal
4. Check for budget conflicts

5. Either pause with options OR complete with local plan
Gemini: NOT called on this route (useGemini: false)

4.5 Route 2: POST /api/agent/respond

Input: { sessionId, userChoice } (e.g. "option_a")

Behavior:

1. Load session from Firestore
 2. Validate status is AWAITING_USER_INPUT
 3. Log user's choice
 4. Build destination-specific itinerary
 5. Optionally call Gemini once for enrichment
 6. Save COMPLETED + finalPlan to Firestore
-

5. The Agent Brain (Core Logic)

5.1 Destination Parser

Extracts city from user goal and returns attraction data.

Built-in cities: Jaipur, Udaipur, Goa, Manali

Generic fallback: Parses phrases like "trip to Varanasi"

Example:

Input: "Plan a trip to Udaipur under 15000"

Output: { name: "Udaipur", state: "Rajasthan", attractions: [...], costs: {...} }

5.2 ReAct Loop (Mock)

Simulates autonomous agent reasoning:

Step	Type	Example Message
1	thought	Decomposing goal...
2	action	Searching destinations and seasonal pricing...
3	observation	Identified Udaipur — lakeside palaces...
4	thought	Estimating transport, lodging, and attraction costs
5	action	Querying attraction schedules and opening hours...
6	observation	Monsoon Palace may exceed budget constraints

In production, these would be real tool calls (flight APIs, hotel pricing, etc.)

5.3 Conflict Detection (Human-in-the-Loop)

If budget "d 15,000, the agent stops autonomously and presents:

- Option A — Stay within budget, skip premium attraction
- Option B — Full experience, accept budget overrun

The API responds immediately at this point. No further agent steps run until the user chooses.

5.4 Plan Builder

Builds a 3-day itinerary from the destination catalog:

Destination	Option A	Option B
Udaipur	City Palace, Lake Pichola (₹14,200) + Monsoon Palace (₹15,800)	
Jaipur	City Palace, Hawa Mahal (₹14,800) + Amber Fort (₹16,500)	
Goa	Baga Beach, Fort Aguada (₹13,500) + Scuba, Dudhsagar (₹17,200)	
Manali	Hadimba Temple, Mall Road (₹14,000) + Rohtang, Paragliding (₹16,800)	

6. Gemini Integration (Optimized)

When Gemini Is Called

Event	Gemini Called?	Reason
Plan Trip (`/run`)	No	Saves API quota
Conflict pause	No	No plan built yet
User picks option (`/respond`)	Yes (max 1 call)	Enrich final plan only
Quota exceeded	No (5-min cooldown)	Circuit breaker
Same goal + choice again	No	In-memory cache

Fallback Chain

Gemini available !' generatedBy: "gemini"
Gemini fails/quota !' generatedBy: "local" (still has summary, highlights, tips)

Optimization Techniques

- 1. Single call per session — only on /respond
- 2. Lightweight model — gemini-2.0-flash-lite
- 3. Token cap — max 200 output tokens
- 4. Compact prompts — minimal context
- 5. JSON response mode — structured output
- 6. In-memory cache — duplicate requests skipped
- 7. Circuit breaker — 5-minute pause after quota errors

7. Frontend Deep Dive (`client/`)

7.1 React State (`App.jsx`)

State Variable	Purpose
`goal`	User's travel request text
`sessionId`	Unique ID per trip attempt
`status`	idle / AWAITING_USER_INPUT / COMPLETED / error
`logs`	Agent activity feed
`pendingOptions`	Option A / B decision cards
`finalPlan`	Completed itinerary object

7.2 API Layer (`api.js`)

Uses fetch() to call backend endpoints. Vite dev server proxies /api/* to localhost:3001 to avoid CORS issues.

7.3 UI Sections

- 1. Input Panel — textarea + "Plan Trip" button
- 2. Agent Activity — live log stream with icons per step type
- 3. Decision Required — conflict reason + clickable option cards
- 4. Your Itinerary — day-by-day plan, cost, highlights, packing tips

8. Environment & Secrets

GEMINI_API_KEY=your_key_here
GEMINI_ENABLED=true
GEMINI_MODEL=gemini-2.0-flash-lite
GOOGLE_APPLICATION_CREDENTIALS=./serviceAccount.json
PORT=3001

Variable	Required	Description
GEMINI_API_KEY	Optional	Google AI Studio API key
GEMINI_ENABLED	Optional	Set `false` to disable LLM
GEMINI_MODEL	Optional	Defaults to flash-lite
GOOGLE_APPLICATION_CREDENTIALS	Required	Path to Firebase service account
PORT	Optional	Defaults to 3001

9. How to Run

Terminal 1 — Backend

```
npm start
# !' http://localhost:3001
```

Terminal 2 — Frontend

```
cd client
npm run dev
# !' http://localhost:5173
```

URLs

URL	Purpose
http://localhost:5173	React UI (main app)
http://localhost:3001/health	Server health check
POST /api/agent/run	Start agent execution
POST /api/agent/respond	Submit user's option choice

10. API Reference

POST /api/agent/run

Request:

```
{
  "userId": "demo-user",
  "sessionId": "session-1234567890",
  "userGoal": "Plan a trip to Udaipur under 15000"
}
```

Response (conflict):

```
{
  "sessionId": "session-1234567890",
  "status": "AWAITING_USER_INPUT",
  "conflictReason": "A lake cruise plus Monsoon Palace...",
  "pendingOptions": [
    { "id": "option_a", "label": "Option A – Stay within budget", "description": "..." },
    { "id": "option_b", "label": "Option B – Full experience", "description": "..." }
  ],
  "logs": [...]
}
```

Response (no conflict):

```
{
  "sessionId": "session-1234567890",
  "status": "COMPLETED",
  "finalPlan": { "destination": "...", "itinerary": [...] },
  "logs": [...]
}
```

POST /api/agent/respond

Request:

```
{
  "sessionId": "session-1234567890",
  "userChoice": "option_a"
}
```

Response:

```
{
  "sessionId": "session-1234567890",
  "status": "COMPLETED",
  "userChoice": "option_a",
  "finalPlan": {
    "destination": "Udaipur, Rajasthan",
    "durationDays": 3,
    "estimatedCostINR": 14200,
    "itinerary": [...],
  }
}
```

```
    "summary": "...",
    "highlights": [...],
    "packingTips": [...],
    "generatedBy": "local"
  },
  "logs": [...]
}
```

11. Production Deployment (Firebase Cloud Functions)

server.js exports the Express app for Cloud Functions:

```
module.exports = app;
```

Deployment Steps

1. Wrap app with functions.https.onRequest(app)
 2. Deploy backend to Firebase Cloud Functions
 3. Deploy React build to Firebase Hosting
 4. Firestore + Admin SDK work automatically in Cloud Functions
 5. Set GEMINI_API_KEY as a Firebase environment secret
-

12. Summary

The Smart Travel Agent is a stateful, human-in-the-loop AI system where:

1. React captures the user's travel goal
2. Express runs a ReAct agent loop with destination-aware logic
3. Firestore persists session state across pauses and resumes
4. Conflict detection pauses autonomous execution for human decisions
5. Plan builder generates city-specific itineraries
6. Gemini optionally enriches the final plan (one call max per session)

The React UI sends a travel goal to Express, which runs a stateful ReAct agent stored in Firestore, pauses for human input when budgets conflict, then builds a destination-aware itinerary — optionally polished by a single Gemini call — and returns the final plan.

Generated for Smart Travel & Route Optimization Agent portfolio project.

